

AI Ops Assignment 2

Madhav Arun Krishna - DA24B012

September 2026

1 Question 1

- The naive single stage Dockerfile: Disk Usage: 725 MB, Content Size: 166 MB
- The multi stage Dockerfile: Disk Usage: 717 MB, Content Size: 164 MB
- Disk Usage Reduction: 1.1%, Content Size Reduction: 1.2%
- Note that in order to be uniform with both my dockerfiles, I chose to use python:3.12-slim in both, and also kept the `--no-cache-dir` argument in the pip install commands of both files.
- For my image, the main differences come from explicitly copying only the needed files into the final docker image as needed for the predictor, i.e. the core predictor.py, the model joblib, and only the required pip packages. The naive one would have included everything else including train.py, the requirements.txt file itself (which is not needed after the requirements are installed) (in the multi stage one, we copy the environment over, so there is no need to copy the requirements file) and other unnecessary files that were in the directory for docker related things.
- This explains the slight savings in space usage.

2 Question 2

- 2.3 Answer:
 - Cache Miss Time: 0.018081s (evidence in evidence folder)
 - Cache Hit Time: 0.004308s (evidence in evidence folder)
 - Hence using the cache has a noticable speedup if we know that we're expecting repeated requests
- 2.4 Answer: A dockerfile just tells docker how to build a single container. Docker Compose helps us orchestrate several docker containers (that may depend on each other) together with a single unified way to manage all of them together on a single machine; you still need dockerfiles to tell docker how to build each container. Kubernetes on the other hand helps you manage several parallel docker containers across machines with custom resource requirements, and automatic self-recovery and easy scaling.

3 Question 3

- 3.1 Answer:
 - If your available cluster has exactly 2 nodes with 2 allocatable CPUs each, then there are a total of 4 CPUs available to us. This naturally makes 4 a very convenient choice, as we can allocate 2 pods to each node giving us a total of 4 pods, each with 1 CPU for itself.
 - This is great, because when doing sequential operations like reading a CSV, it would be easier to just have 1 CPU per pod, and we don't need to split any CPUs across pods. Since there are 4 CPUs available and each pod requests 1 CPU, at most 4 pods can run at the same time. The remaining completions can run once the first set of pods finishes.
 - The Downward API fields are metadata.name and spec.nodeName. These can be made available to the container as environment vars (in my repo `POD_NAME` and `NODE_NAME`)

- 3.2 Answer: Shared in evidence images and the video recording.
- 3.3 Answer:
 - I used the Kubernetes API to read the pod logs rather than using a shared volume. Minikube's default storage provisioner provides host-backed storage rather than a true shared multi-node filesystem, so using pod logs avoids unnecessary coordination for these small results.
 - The invalid row counts were (Total rows were 100 in each, and my generator was generating wrong rows with a probability of 50%):
 - * user_signup_data_1.csv: 48
 - * user_signup_data_2.csv: 53
 - * user_signup_data_3.csv: 42
 - * user_signup_data_4.csv: 58
 - * user_signup_data_5.csv: 53
 - * user_signup_data_6.csv: 49
 - * user_signup_data_7.csv: 37
 - * user_signup_data_8.csv: 55
- 3.4 Answer:
 - If the cluster instead had 3 nodes with 2 CPUs each, then there would be a total of 6 CPUs available, my decision would depend on the problem.
 - For this exact problem (as it is, with exactly 8 CSV shards, and every CSV shard having roughly the same size), I would keep the parallelism as 4 to avoid wasting unnecessary resources of the cluster. Doing 6 parallelly wouldn't really change the total speed to complete the job (as you would still need 2 waves to complete it), but would unnecessarily involve 6 cpus. I would just avoid using the third node as it doesn't help our case
 - If the problem were more general, say a random number of CSV shards, or perhaps shards with different sizes, there is a possibility of increased speed up and I would increase the parallelism to 6, since there are 6 CPUs available.

4 Question 4

- 4.2 Answer: My deployment is maintaining a ReplicaSet with a desired replica count of 2. When I manually deleted one of the running pods (shown in video and evidence images), the ReplicaSet controller was able to detect that the number of pods present was below the desired number and created a replacement pod automatically, very quickly.
- 4.3 Answer: If this application had been deployed as a Job instead of a Deployment, Kubernetes would be considering those pods as tasks meant to finish on completion rather than as a service that was just always continuously available. A Job would not do the same rolling update from v1 to v2 which we saw while keeping the service available. Instead, the Kubernetes Deployment just slowly replaced the old pods with new ones using the RollingUpdate strategy we had set up.
- 4.4 Answer:
 - A Deployment is better suited for our spam detection API because we want it to be a continuously hosted service that users can send requests to. There is no 'task' that needs to be finished. So, we would want to just keep the required number of nodes/replicas running to support our usage demands, while also supporting rolling updates without downtime.
 - A Job is better suited for Question 3 because there was a fixed task that needed completion, which was validating all the given CSVs. It was a finite task with a clear end goal, and no user expectations after that goal was met. Thus, we could just have a system where each pod processes its shard, reports the result, and then completes.